FastAPI Study Notes

What FastAPI Is

FastAPI is a modern, high-performance Python web framework for building APIs. It is built on top of Starlette for the web parts and Pydantic for data validation. Its headline features are automatic interactive documentation, native async support, and type-hint-driven validation that reduces boilerplate.

Route Decorators

Endpoints are declared with decorators on an APIRouter or the app instance, such as @app.get("/items") or @router.post("/items"). The decorated function's parameters and return type annotations define how the request is parsed and how the response is serialized. Path and query parameters are inferred from the function signature.

Dependency Injection

FastAPI provides a powerful dependency injection system via the Depends() helper. Dependencies are plain callables that can supply shared resources such as a database session, the current user, or configuration. They are resolved per request and can themselves depend on other dependencies, forming a clean, testable graph.

Pydantic Models

Request and response bodies are described with Pydantic models. FastAPI uses these models to validate incoming JSON, coerce types, and produce clear validation errors automatically. The same models drive the generated OpenAPI schema, keeping documentation in sync with the code.

Async Support

Route handlers can be defined with async def, allowing the use of await for non-blocking I/O such as database queries or external HTTP calls. Because FastAPI runs on an ASGI server like Uvicorn, async handlers let a single worker serve many concurrent requests efficiently. Synchronous handlers are still supported and are run in a thread pool.

Validation and Errors

When validation fails, FastAPI returns a 422 response with a structured list of the offending fields. Application code can raise HTTPException to return specific status codes and detail messages, which is the idiomatic way to surface business errors such as 404 Not Found or 429 Too Many Requests.

Why It Suits This Project

MiniGlean uses FastAPI for its async database access (SQLAlchemy with asyncpg), its streaming responses for the chat endpoint, and the automatic documentation that makes the API easy to explore during development.